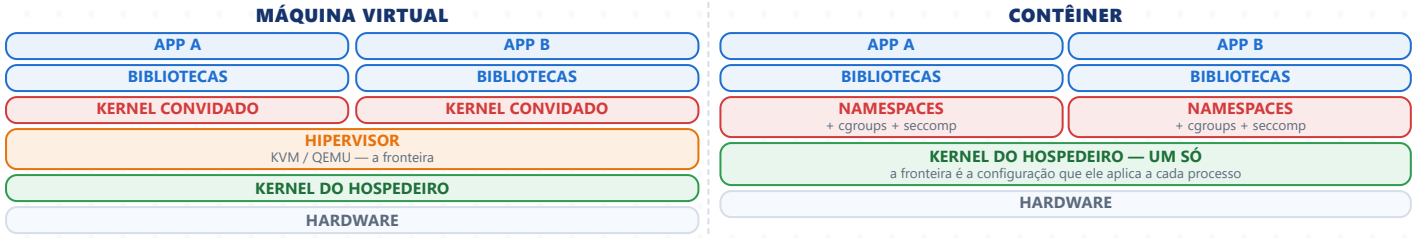
GUIA VISUAL 099

CONTÊINERES POR DENTRO: NAMESPACES, CGROUPS E DOCKER

*Dentro do contêiner, `ps` mostra dois processos, `hostname` responde `a3f9c2b1e004` e `ls /` mostra uma árvore que não é a sua. No hospedeiro, o mesmo software é o PID 48211 na saída de `ps`. **As duas visões estão certas:** contêiner não é VM em miniatura — é processo comum com a visão do sistema recortada.*

099
GUIAS LINUX

ONDE CADA TECNOLOGIA COLOCA A FRONTEIRA DE ISOLAMENTO

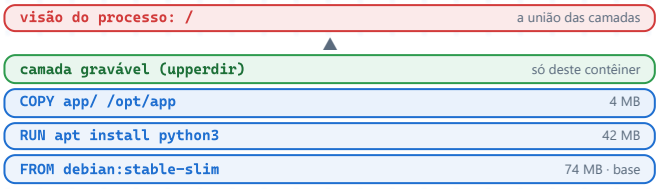


À esquerda, cada convidado tem kernel próprio: quem for comprometido lá dentro ainda precisa furar o hipervisor para chegar ao hospedeiro. À direita, o kernel é um só. Contêiner sobe em milissegundos e custa pouca memória; VM leva segundos e reserva RAM. **A pergunta não é "qual é melhor", e sim "de quem preciso me defender":** contêiner separa aplicações que *você* controla; VM separa código em que você *não confia*. E como o kernel é um só, **falha explorável de dentro do contêiner alcança o hospedeiro e todos os outros**.

AS TRÊS PEÇAS: RECORTAR O QUE ELE VÊ, LIMITAR O QUE CONSOME, EMPILHAR O QUE ELE LÊ

NAMESPACE	RECORTA · EFEITO VISÍVEL
`mnt`	tabela de montagem — outra árvore de diretórios
`pid`	numeração — o primeiro processo é o **PID 1**
`net`	interfaces, rotas e firewall próprios
`uts`	`hostname` responde outra coisa
`user`	mapeia UID/GID: **root dentro, usuário comum fora**
`cgroup`	não vê a hierarquia de cgroups do hospedeiro
`ipc · time`	filas e memória compartilhada não atravessam; relógio e uptime próprios

São independentes — e o Docker *não* usa o `user` por padrão. **Dois processos estão no mesmo namespace se, e só se, o número em `/proc/<PID>/ns/` coincidir.**



As camadas de baixo são somente-leitura e compartilhadas por todos os contêineres da mesma base — dez contêineres ocupam o espaço de um. Ao gravar num arquivo que vem de baixo, o overlayfs faz *copy-up*: copia para a camada gravável e altera a cópia. **Removida ela, tudo o que o contêiner escreveu desaparece** — daí a regra: **dado que precisa durar vai em volume**, nunca na camada gravável. Cada instrução do `Dockerfile` vira uma camada: agrupe `RUN` e ordene do menos para o mais volátil — é assim que o cache de build rende.

cgroups v2 são diretórios em `/sys/fs/cgroup/`, e impor limite é escrever num arquivo: `memory.max` (estourar aciona o OOM killer *dentro* do cgroup), `pids.max` (faz `fork` falhar — a defesa contra *fork bomb*) e `cpu.max`. O systemd já gerencia essa árvore — cada unit é um cgroup, daí `systemd-cgtop`.

NA PRÁTICA — CONSTRUIR NA MÃO E RECONHECER AS PEÇAS NA CLI DO DOCKER

kali@kali: ~

```
# arvore de processos propria (namespace PID)
$ sudo unshare --pid --fork --mount-proc bash
# ps aux   # dentro: bash e PID 1, 2 processos
# sem --mount-proc o isolamento existe no kernel e
# NAO aparece: ps le o /proc do hospedeiro
# root que nao e root (namespace user, sem sudo!)
$ unshare --user --map-root-user bash
# id
uid=0(root) gid=0(root)
# cat /proc/self/uid_map
    0    1000    1  # root aqui = 1000 la fora
# laboratorio de rede inteiro sem uma VM sequer
$ sudo ip netns add lab
$ sudo ip link add veth-h type veth peer name veth-c
$ sudo ip link set veth-c netns lab   # cabo virtual
```

OPÇÃO DO DOCKER RUN	QUAL MECANISMO ELA EXPÕE
`--memory --cpus --pids-limit`	cgroups v2: `memory.max`, `cpu.max`, `pids.max`
`--cap-drop=ALL · no-new-privileges`	capabilities e escalada por SUID; readicione só o que faltar
`--read-only --tmpfs /tmp · USER app`	a camada gravável e as montagens; **não rode como root** lá dentro
`docker inspect .State.Pid`	com `ls -l /proc/<PID>/ns/`, prova que o contêiner **é** aquele processo
`podman (sem daemon)`	filho do *seu* usuário, com o namespace `user`: *rootless* por construção

Três opções anulam o isolamento, cada uma por si um caminho direto para o hospedeiro. `--privileged` devolve todas as capabilities, remove o seccomp e abre `/dev` — um processo assim monta o disco do hospedeiro e edita o `/etc/shadow`; prefira `--cap-add` específico. `--net=host` remove o namespace de rede: ele passa a ver e capturar todo o tráfego da máquina. **O socket do Docker montado** dá ao contêiner o poder de criar *outros* contêineres, inclusive um `--privileged` montando `/` — equivale a dar root no hospedeiro. **Pela mesma lógica, estar no grupo `docker` equivale a ter root sem senha:** `docker run -v /:/hosp -it debian chroot /hosp` é tudo o que separa um membro do grupo do controle total da máquina.

RESUMO RÁPIDO

- ✓ **Contêiner é processo, não máquina.** Namespaces recortam o que ele *vê*, cgroups limitam o que ele *consome*, capabilities/seccomp/MAC restringem o que ele *pode fazer*. Sem kernel próprio, sem hardware emulado.
- ✓ **Oito namespaces independentes** — `mnt`, `pid`, `net`, `uts`, `ipc`, `user`, `cgroup`, `time`. `unshare` cria, `nsenter` entra (e resgata contêiner sem shell dentro, com os binários do hospedeiro), `lsns` lista.
- ✓ `--mount-proc` **junto com** `--pid` **não é detalhe:** sem ele o isolamento existe no kernel e as ferramentas continuam mostrando a máquina inteira.
- ✓ **Imagem é pilha somente-leitura sob uma camada gravável**, unidas por overlayfs. Dado que precisa durar vai em **volume**.
- ✓ `--privileged`, `--net=host` **e o socket do Docker montado anulam o isolamento**; grupo `docker` = root sem senha. Podman *rootless* evita a categoria inteira.
- ✓ **Contêiner isola menos porque o kernel é compartilhado:** aplicação sua em contêiner, código não confiável em VM.

NO KALI

Contêiner é o jeito mais barato de manter o Kali limpo. Em vez de empilhar dezenas de ferramentas com dependências conflitantes no sistema principal, `docker run --rm -it kalilinux/kali-rolling` dá um Kali descartável, e um `apt install kali-linux-headless` lá dentro monta o ambiente completo que você joga fora depois — cada ferramenta na sua imagem. É a aplicação prática dos dotfiles: a VM some, o `Dockerfile` fica. **Alvos de laboratório (DVWA, Juice Shop, WebGoat) sobem com um comando — com uma ressalva séria:** publique a porta *só no loopback*, `-p 127.0.0.1:8080:80`, nunca `-p 8080:80`, e jamais em máquina com IP público. Aplicação deliberadamente vulnerável exposta à internet é comprometida em minutos e vira um problema seu, não didático. Alvo que exija kernel próprio ou escalada local pede VM isolada. **Ferramenta de rede em contêiner ensina o modelo na marra:** `nmap -sS`, `tcpdump` e `ping` pedem `CAP_NET_RAW`, e a resposta é `--cap-add=NET_RAW`, nunca `--privileged`. Falhou com "operation not permitted"? A pergunta é *qual capability* ou *qual syscall barrada pelo seccomp* — o `strace` responde. **Do lado defensivo, contêiner mal configurado é hoje um dos achados mais comuns em teste de infraestrutura.** A lista que se verifica primeiro: rodando como root, `--privileged` sem justificativa, socket do Docker montado, imagem base velha, segredo em `ENV` ou numa camada (`docker history` revela) e a API na porta `2375` sem TLS — root remoto sem autenticação. `trivy` e `docker scout` automatizam a varredura da imagem. Só em sistema próprio ou autorizado: invadir é crime (Lei 12.737/2012).